Student Workbook

Advanced JavaScript: Functions, `this`, Classes, and Prototypes

Duration: 3-hour required core, including two 10-minute breaks, inside a 4-hour class window **Goal:** Build one connected mental model for JavaScript behavior, from function values to prototype lookup.

Before class

You should be comfortable declaring variables, calling functions, using arrays and loops, and creating a basic object literal. You do not need prior experience with callbacks, arrow functions, classes, inheritance, or prototypes.

The default classroom setup is a current desktop browser. You do not need Node.js or `npm`.

Before class, open a new browser tab, enter `about:blank`, and open the JavaScript console:

- Chrome or Edge on macOS: **Command-Option-J**
- Chrome or Edge on Windows or Linux: **Control-Shift-J**
- Firefox on macOS: **Command-Option-K**
- Firefox on Windows or Linux: **Control-Shift-K**

Click the **Console** tab if necessary. Type `1 + 1` next to the `>` prompt and press **Enter**. If you see `2`, you are ready.

During class, copy one `js` code block at a time into the console and press **Enter**. If you type a multiline example by hand, use **Shift-Enter** to start a new line without running the code yet.

Two console behaviors are useful to know:

1. Seeing `undefined` after output from `console.log` is normal. It is usually the return value of `console.log`, not an error.
2. Some examples build on the preceding code block, so reload only when your instructor says to start fresh. If you see “Identifier has already been declared,” reload the blank page and rerun that example section from its beginning. Clearing the visible console output does not reset declared variables; reloading does.

For each lab, make your changes in the matching file under `exercises/`, then copy that code into a freshly reloaded console to try it. Your instructor will walk through this process before the first exercise.

Learning outcomes

By the end, you will be able to:

1. treat a function as a value that can be stored, passed, and returned;
2. explain the difference between `doSomething` and `doSomething()`;
3. recognize callbacks and higher-order functions;
4. translate between regular function expressions and arrow functions;
5. choose a regular method or an arrow callback based on `this` behavior;
6. create independent instances with a class and constructor;
7. distinguish instance properties and methods from static properties and methods;
8. trace a missing property through an object's prototype chain.

The course map

```text
Functions are values
        |
        v
Functions can be stored, passed, and returned
        |
        v
Callbacks supply behavior to higher-order functions
        |
        v
Arrow functions make small callbacks lightweight
        |
        v
Objects combine state and behavior
        |
        v
Classes create related objects with shared behavior
        |
        v
this identifies the object a regular method is working with
        |
        v
Prototypes explain how behavior is shared and inherited
```

Part 1: Functions as Values

1. First-class functions

Start with a familiar declaration:

```
js
function greet(name) {
  return `Hello, ${name}`;
}
```

Compare these two expressions:

```
js
greet("Maya");
greet;
```

Write your prediction before running the code.

- `greet("Maya")` evaluates to: _____
- `greet` evaluates to: _____

The parentheses are an operation: they call the function. Without parentheses, `greet` is the function value itself.

Checkpoint

```
js
const action = greet;
```

What does `action` contain?

- [ ] The string returned by The string returned by `greet`
- [ ] The function itself
- [ ] A second, unrelated copy of the function

What will this produce?

```
js
action("Alex");
```

Prediction: _____

Lab 1: Select a function

Open `exercises/01-function-values.js`.

Your goal is to make one function choose and return another function. The returned function should not run until the caller later uses parentheses.

When finished, answer:

1. Which line selects the function value? _____
2. Which line calls that function? _____
3. Why would `selectGreeting(true)("Maya")` also work? _____

---

2. Storing and passing functions

Functions can be stored anywhere other values can be stored.

```
js
const robot = {
  name: "R2",
  move() {
    return "Moving";
  }
};
```

When a function is stored on an object and called through that object, we usually call it a **method**.

```
js
console.log(robot.move());
```

A function can also be supplied as an argument:

```
js
function run(action) {
  return action();
}

function jump() {
  return "Jump!";
}

console.log(run(jump));
```

Prediction pair

Complete the table before discussing it.

Expression	What happens before `run` starts?	What value is passed?
`run(jump)`		
`run(jump())`		

If a one-use function does not need a reusable name, it can be written in place:

```
js
console.log(run(function () {
  return "Skip!";
}));
```

This is an **anonymous function expression**. It is not mysterious new behavior; it is simply a function value created at the point where another function expects a value.

---

3. Callbacks and higher-order functions

A **callback** is a function supplied to other code so that code can call it at the appropriate time.

A **higher-order function** accepts a function, returns a function, or both.

```
js
function repeat(count, action) {
  for (let index = 0; index < count; index += 1) {
    action(index);
  }
}

repeat(3, function (index) {
  console.log(index);
});
```

Label the roles:

- Higher-order function: _____
- Callback: _____
- Argument supplied to the callback each time: _____

Array methods use the same pattern:

```
js
const scores = [72, 91, 84, 65, 98];

const highScores = scores.filter(function (score) {
  return score >= 80;
});
```

The browser example in `demos/03-button.html` uses the same relationship: `addEventListener` receives a callback and calls it later when the button is clicked.

Read this as:

> Filter performs the traversal. The callback describes the keep-or-discard decision for one item.

Pattern, not catalog

```
js
players.filter(isAlive);
players.map(getScore);
players.sort(compareScores);
```

In each expression, underline the **operation** and circle the **behavior passed into it**.

---

4. Arrow functions

These three callbacks behave the same for this filtering task:

```
js
scores.filter(function (score) {
  return score >= 80;
});

scores.filter((score) => {
  return score >= 80;
});

scores.filter(score => score >= 80);
```

Syntax reference

```
js
() => console.log("Go");        // zero parameters
x => x * 2;                     // one parameter, implicit return
(x, y) => x + y;                // multiple parameters
x => {                          // block body
  const doubled = x * 2;
  return doubled + 1;
};
name => ({ name, score: 0 });   // object literal in parentheses
```

Two rules prevent most early arrow-function bugs:

1. A block body `{ ... }` needs an explicit `return` when it should produce a value.
2. An implicitly returned object literal must be wrapped in parentheses: `value => ({ value })`.

Lab 2: Build a callback pipeline

Open `exercises/02-callback-pipeline.js`.

Complete the three arrow callbacks so the result is:

```
js
["MAYA", "SAM"]
```

For each callback, write its contract:

Operation	Callback receives	Callback should return
`filter`		
first `map`		
second `map`		

Before break: one-minute retrieval

Without looking back, complete these sentences:

- A callback is _____.
- A higher-order function _____.
- `greet` means _____, while `greet()` means _____.
- Arrow functions are especially convenient when _____.

Part 2: `this`, Objects, and Classes

5. Regular functions, arrows, and `this`

For the method call below, `this` refers to the object to the left of the dot at call time:

```
js
const player = {
  name: "Maya",
  introduce() {
    return `I'm ${this.name}`;
  }
};

player.introduce();
```

The practical model is about the **call**, not where a function was first written.

```
text
player.introduce()
^^^^^^
this for this regular method call
```

Arrow functions behave differently: they do not create a new `this`. They use `this` from the surrounding scope.

This makes an arrow a poor default for an object method that needs the object:

```
js
const arrowMethod = {
  name: "Alex",
  introduce: () => `I'm ${this.name}`
};
```

In a Node.js ES module, the surrounding top-level `this` is `undefined`, so attempting to read `this.name` throws a `TypeError`. In a classic browser script, the arrow may capture `window` instead. In neither case does the arrow receive `arrowMethod` as its method-call receiver.

But it makes an arrow useful inside a regular method:

```
js
const team = {
  name: "Longhorns",
  players: ["Maya", "Alex", "Sam"],

  printPlayers() {
    this.players.forEach(player => {
      console.log(`player < /span > playsfor < spanclass = "hljs − subst" >{this.name}`);
    });
  }
};
```

The outer regular method gets `this` from `team.printPlayers()`. The inner arrow keeps that same `this`.

Practical choice rule

- Use regular method syntax when a method needs `this` from the receiver.
- Use an arrow for a short callback that should keep the surrounding `this`.
- Choose based on behavior, not only character count.

Lab 3: Repair `this`

Open `exercises/03-this-and-arrows.js`.

Fix the two function choices without referring to the object by its variable name inside `print`. Explain why each change is appropriate:

- Outer method: _____
- Inner callback: _____

---

6. Object literals as a bridge

An object literal creates one object directly:

```
js
const player = {
  name: "Maya",
  score: 0,
  addPoint() {
    this.score += 1;
  }
};
```

That is often exactly what a program needs. But if Maya, Alex, Sam, and Jordan all need the same structure and behavior, repeated object literals invite duplication. A class describes a category of objects that we can create repeatedly.

---

7. Classes, constructors, and instances

```
js
class Player {
  constructor(name) {
    this.name = name;
    this.score = 0;
  }

  addPoint(points = 1) {
    this.score += points;
  }

  describe() {
    return `<span class="hljs-variable language">this</span>.name</span>:<sp
  }
}
```

Three pieces:

Syntax	Mental model
`class Player`	describes a category of related objects
`constructor(...)`	establishes initial state for each new object
`new Player(...)`	creates and initializes an instance

```
js
const maya = new Player("Maya");
const alex = new Player("Alex");
```

Prediction

```
js
maya.addPoint();
maya.addPoint();

console.log(maya.score);
console.log(alex.score);
```

- `maya.score` : _____
- `alex.score` : _____
- Why? _____

Lab 4: Build `Robot`

Open `exercises/04-robot-class.js`.

Implement the constructor and instance methods described in the file. Create both `wallE` and `robbie`, then make each robot greet, charge, and complete its own tasks.

Before moving on, verify:

- [ ] Every instance owns its own `name`, `color`, `battery`, and `tasksCompleted` Every instance owns its own `name`, `color`, `battery`, and `tasksCompleted` properties.
- [ ] `greet()` reads the receiving robot's name and color through `this` `greet()` reads the receiving robot's name and color through `this`.
- [ ] `charge(amount)` `charge(amount)` increases only the receiving robot's battery and caps it at 100.
- [ ] `doTasks(tasks)` `doTasks(tasks)` records every task on the receiving robot.
- [ ] Wall-E's tasks do not appear in Robbie's `tasksCompleted` Wall-E's tasks do not appear in Robbie's `tasksCompleted` array.

---

8. Instance properties, methods, and `this`

```
js
maya.score;       // state for this instance
maya.addPoint();  // shared behavior operating on this instance
```

The same method works with different receivers:

```
text
maya.addPoint()  -> this = maya
alex.addPoint()  -> this = alex
```

Complete the sentence:

> `this` lets one shared method _____.

A useful observation

Predict these results:

```
js
maya === alex
maya.addPoint === alex.addPoint
```

- First result: _____ because _____
- Second result: _____ because _____

The second answer points toward prototypes: instances can have different state while sharing the same method value.

---

9. Static properties and methods

A static property belongs to the class itself, not to one instance. A static method is a static property whose value is a function.

```
js
class Player {
  static playerCount = 0;

  constructor(name) {
    this.name = name;
    this.score = 0;
    Player.playerCount += 1;
  }

  static getPlayerCount() {
    return Player.playerCount;
  }
}
```

```
js
const maya = new Player("Maya");

maya.name;                // instance
Player.getPlayerCount();  // class
```

Classify each call:

Expression	Instance or static?	What is it about?
`maya.addPoint()`		
`Player.getPlayerCount()`		
`Array.isArray(values)`		
`Object.keys(player)`		

Lab 5: Add Robot fleet properties

Open `exercises/05-robot-static.js`.

Add `Robot.robotCount` and increment it whenever the constructor creates a robot. Then add `Robot.describeFleet()` and call it on the class—not on `wallE` or `robbie`.

Verify:

- [ ] After two constructor calls, `Robot.robotCount` is `2` After two constructor calls, `Robot.robotCount` is `2`.
- [ ] `Robot.describeFleet()` `Robot.describeFleet()` reports two robots.
- [ ] `wallE.describeFleet` is `undefined` `wallE.describeFleet` is `undefined` because the static method is not an instance property.

Part 3: Prototypes and Lookup

10. Where are properties stored?

Both expressions work:

```
js
maya.name;
maya.addPoint();
```

But the properties do not have to live in the same place.

```
js
Object.hasOwn(maya, "name");      // true
Object.hasOwn(maya, "addPoint");  // false
```

Class method syntax places `addPoint` on `Player.prototype`.

```
js
Object.getPrototypeOf(maya) === Player.prototype; // true
```

Lookup chain

```text
maya
  |
  v
Player.prototype
  |
  v
Object.prototype
  |
  v
null
```

For `maya.addPoint`, JavaScript conceptually asks:

1. Does `maya` have its own `addPoint` property?
2. If not, does `Player.prototype` have it?
3. If not, continue to the next prototype.
4. If the chain reaches `null`, the property is missing and lookup produces `undefined`.

Shadowing prediction

```js
maya.describe = function () {
  return "Special Maya";
};
```

Which implementation will `maya.describe()` use now, and why?

What will `alex.describe()` use?

An own property found earlier in the chain **shadows** a same-named property farther up the chain.

---

11. Basic inheritance

```
js
class SuperPlayer extends Player {
  doubleScore() {
    this.score *= 2;
  }
}
```

The instance lookup chain is now longer:

```
text
superPlayer
    |
    v
SuperPlayer.prototype
    |
    v
Player.prototype
    |
    v
Object.prototype
    |
    v
null
```

`doubleScore` is found on `SuperPlayer.prototype`; inherited `addPoint` is found later on `Player.prototype`.

Lab 6: Prototype detective

Open `exercises/06-prototype-detective.js`.

Complete `SpaceRobot` by calling `super(...)`, storing `homePlanet`, tracking `numSpaceRobots`, and adding `makeCoffee(temp)`. Before running the file, predict every printed Boolean and then verify your answers.

For each successful lookup, record where the property was found:

Expression	Found on
`c3po.name`	
`c3po.charge`	
`c3po.makeCoffee`	
`c3po.toString`	

Final Retrieval and Exit Ticket

Complete the model

Use each term once: **arrow function**, **callback**, **class**, **first-class**, **higher-order function**, **instance**, **prototype**, **static**, **this**.

1. JavaScript functions are _____ values, so they can be stored and passed.
2. A function supplied for another operation to call is a _____.
3. A function that accepts or returns another function is a _____.
4. A short _____ is often convenient when surrounding `this` should be preserved.
5. In a regular method call, _____ identifies the receiver for that call.
6. A _____ describes related objects that can be created repeatedly.
7. An object created with `new` is an _____.
8. A _____ property belongs to the class rather than one instance.
9. A missing property can be sought on the object's _____ chain.

Trace this code

```
js
class Counter {
  static created = 0;

  constructor(label) {
    this.label = label;
    this.value = 0;
    Counter.created += 1;
  }

  increment() {
    this.value += 1;
  }
}

const left = new Counter("left");
const right = new Counter("right");
left.increment();
```

Write the result and one-sentence explanation for each:

1. `left.value` -> _____
2. `right.value` -> _____
3. `Counter.created` -> _____
4. `Object.hasOwn(left, "value")` -> _____

5. `Object.hasOwn(left, "increment")` -> _____
6. `left.increment === right.increment` -> _____

Exit ticket

1. In one sentence, distinguish `functionName` from `functionName()`.
2. Give one situation where an arrow is a strong choice and one where it is not.
3. Explain how two instances can have different state while using the same method.
4. Circle the topic you want to practice next: callbacks / arrows / `this` / classes / prototypes.

Optional Flex Extension: Closures and Promises

Use this section only if your instructor includes the optional flex material.

A closure keeps surrounding data available

```
js
function makeMinimumScore(minimum) {
  return score => score >= minimum;
}

const isPassing = makeMinimumScore(80);
console.log(isPassing(84)); // true
console.log(isPassing(65)); // false
```

`makeMinimumScore(80)` returns a function value. That returned function keeps access to the `minimum` value from the call that created it. This lasting connection between a function and its surrounding lexical environment is a **closure**.

A promise supplies a later value to callbacks

```
js
Promise.resolve({ name: "Maya", score: 91 })
  .then(player => player.score)
  .then(score => console.log(`Score: ${score}`));
```

`then` accepts a callback. The promise controls when that callback runs, and the callback describes what to do with the available value. The first callback turns a player into a score; the second receives that score.

The foundation is the same material used throughout the course: functions are values, higher-order functions accept callbacks, callbacks return values, and arrows provide concise syntax when their `this` behavior is appropriate.

After class

Revisit the examples in this workbook one code block at a time in a freshly reloaded browser console. Before running each block, predict what it will do; afterward, explain the result aloud.

If you already have Node.js 20 or later and want to run the complete self-checking files, open a terminal in the course-materials folder and run them directly:

```
bash
node --version
node demos/01-first-class-functions.js
node demos/02-storing-passing-anonymous.js
node demos/03-callbacks-higher-order-functions.js
node demos/04-arrow-functions.js
node demos/05-arrow-functions-and-this.js
node demos/06-objects-to-classes.js
node demos/07-instances-methods-this.js
node demos/08-static-properties.js
node demos/09-prototypes-and-inheritance.js
```

No `npm install` step is needed. The six matching solution files can be run the same way, for example `node solutions/01-function-values.js`.

Then revisit each starter exercise from a blank file and explain every use of parentheses, `this`, `static`, and `extends` aloud. If you can predict a lookup before running the code, you are reasoning from the model rather than memorizing syntax.